TEAM-14

Code-Judge System Architecture

The architecture for Code-Judge is built on a Microservices model designed for high scalability and secure, isolated code execution. It utilizes Docker as the primary engine for sandboxing untrusted code.

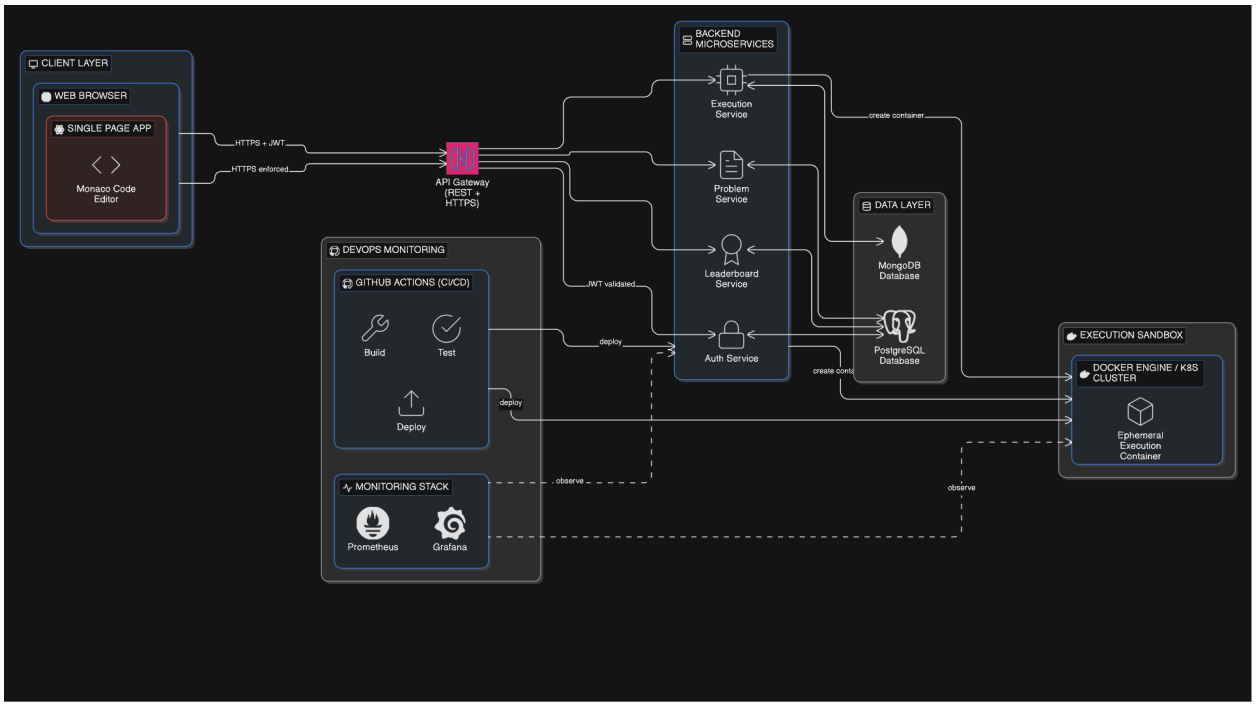
Architectural Components

API Gateway: Acts as the single entry point for all client requests, managing routing and rate limiting.
User Interface (Frontend): A web-based Single Page Application (SPA) that communicates with the backend via JSON requests.
Execution Engine:The critical service that interfaces with the Docker Daemon via Unix Sockets or APIs to lifecycle-manage ephemeral sandboxes.
Data Persistence: PostgreSQL: Stores relational data like User profiles and Problem sets.
MongoDB:Manages unstructured data such as submission logs and source code.
Message Broker: Uses an asynchronous broker (like RabbitMQ) to queue code submissions for processing.
CI/CD Pipeline: Integrated with GitHub Actions to automate the building and deployment of microservices.

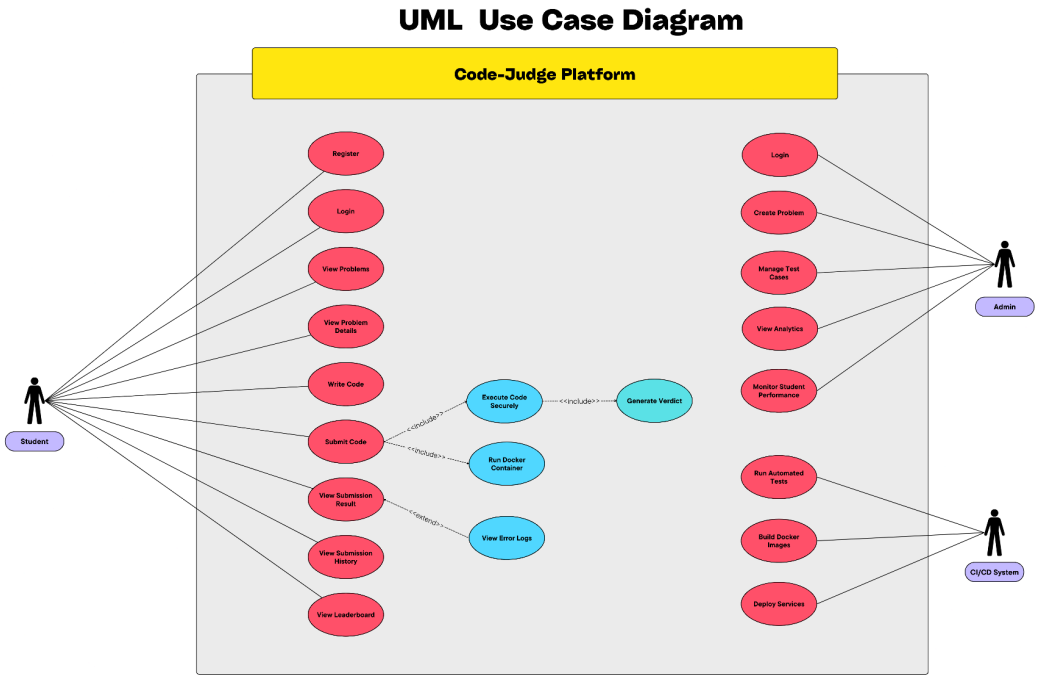
Security & Isolation (Sandboxing)

To maintain host integrity, the architecture enforces a Sandboxed Execution Model:
Containerization: All user code runs inside isolated Docker containers.
Resource Throttling: The engine strictly limits CPU and memory usage to prevent system crashes.
Network Blocking: Submission containers have network access disabled to prevent data exfiltration.
Privilege Control: Processes inside containers run as a non-root user (UID > 1000).

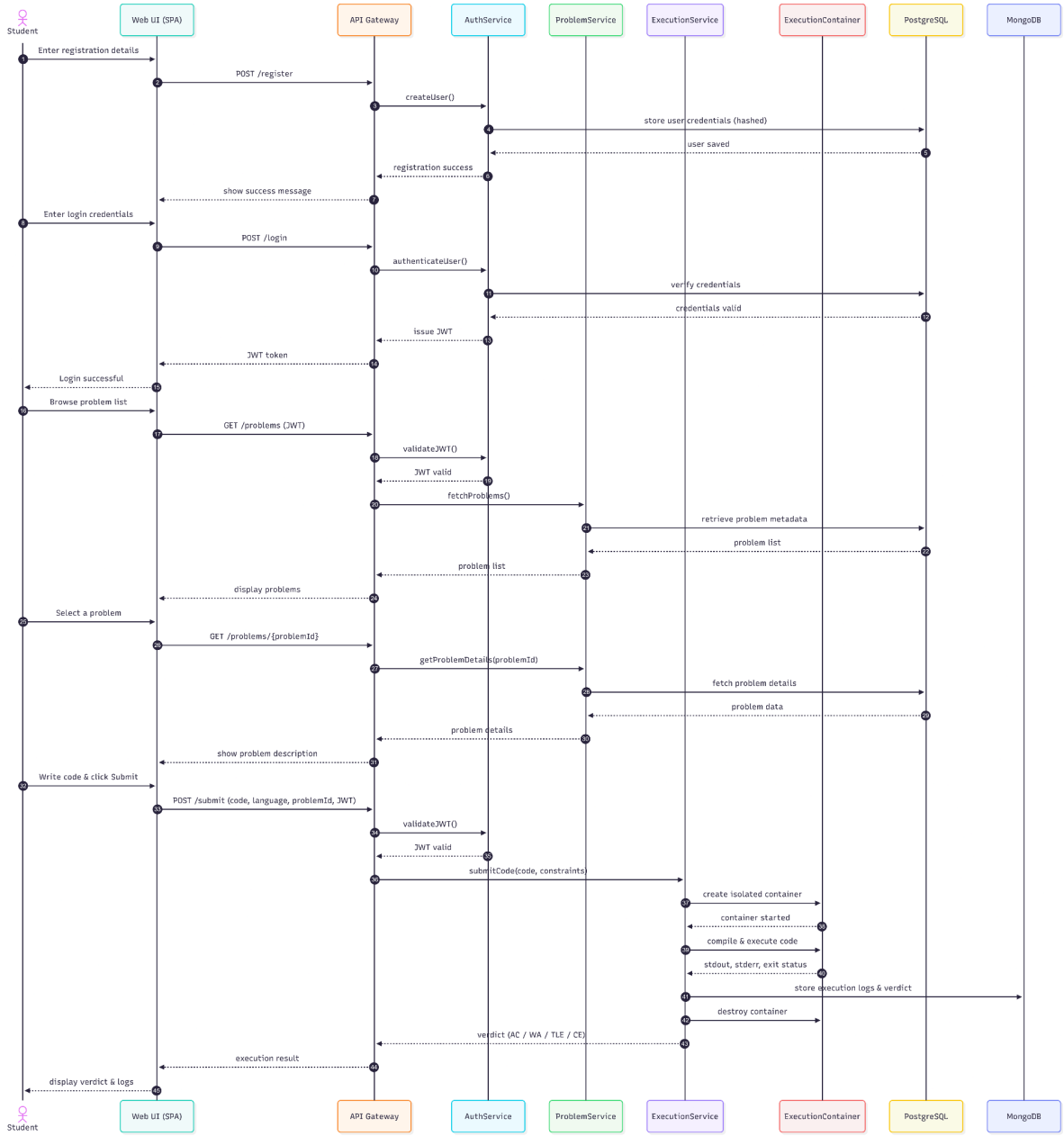
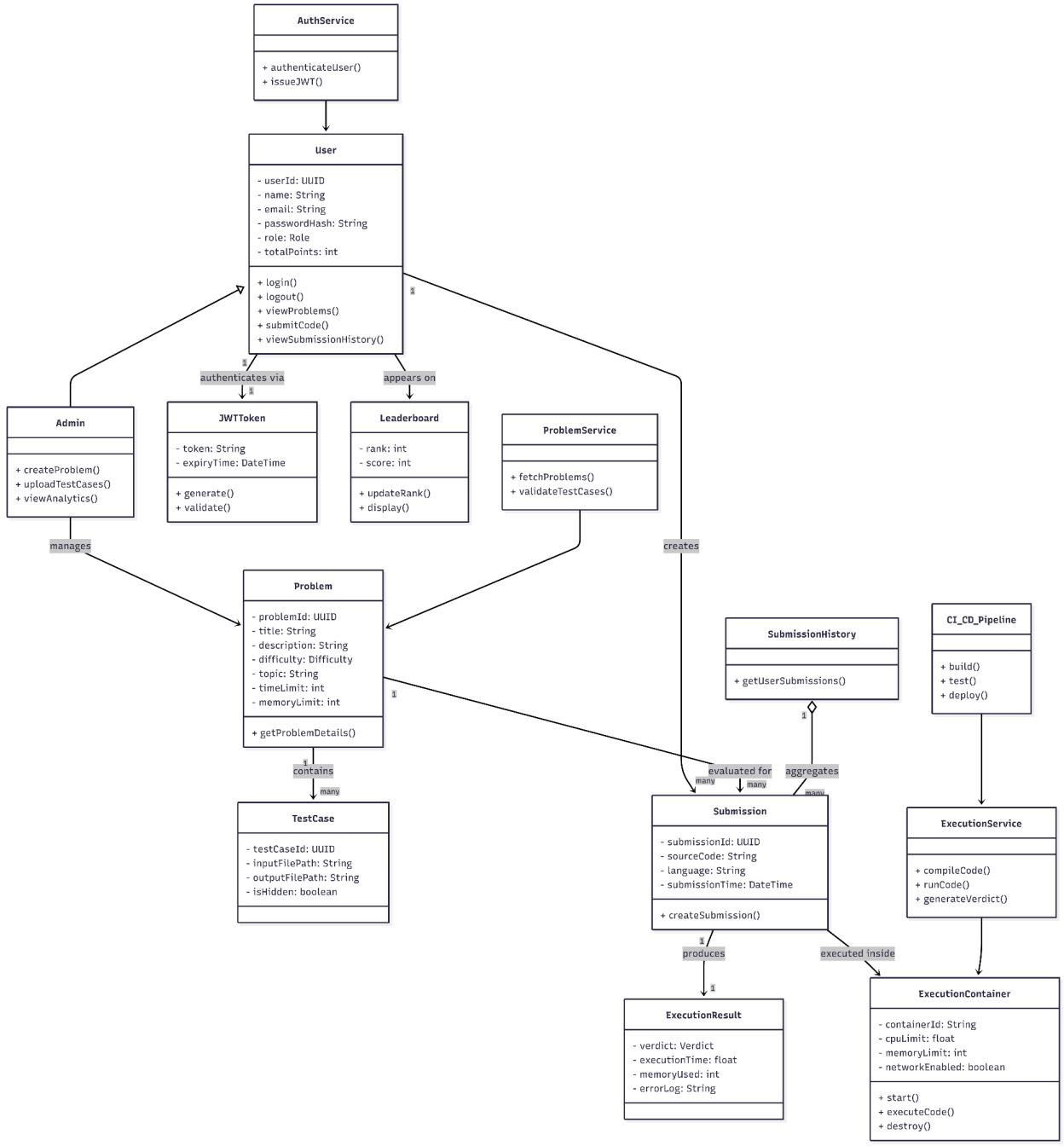
ARCHITECTURE DIAGRAM:



UML DIAGRAM:



SEQUENCE DIAGRAM:



CLASS DIAGRAM: